The Distance-Vector (DV) Routing Algorithm

1 Overview: Distance-Vector (DV) Approach

Unlike the Link-State (LS) algorithm which uses global information, the **Distance-Vector (DV)** algorithm is decentralized and operates based on local information exchange.

Core Characteristics of DV

- **Distributed:** Each node receives info from neighbors, calculates results, and distributes them back.
- **Iterative:** The process continues until no more information is exchanged (self-terminating).
- **Asynchronous:** Nodes do not need to operate in lockstep.

2 The Bellman-Ford Equation

The DV algorithm is mathematically rooted in the **Bellman-Ford Equation**. It defines the least-cost path relationship between a node and its neighbors.

The Bellman-Ford Equation

Let $d_x(y)$ be the cost of the least-cost path from node x to node y. The equation states:

$$d_x(y) = \min_v \{c(x,v) + d_v(y)\}$$

where the minimum is taken over all neighbors v of node x.
Intuition: To get from x to y, you must first go to a neighbor v (cost $c(x,v)$) and then take the shortest path from v to y (cost $d_v(y)$).

Quick Verification

In a network where source u has neighbors v, x, w:

- If $d_v(z) = 5, d_x(z) = 3, d_w(z) = 3$
- And link costs are $c(u,v) = 2, c(u,x) = 1, c(u,w) = 5$
- Then: $d_u(z) = \min\{2+5, 1+3, 5+3\} = \min\{7,4,8\} = \mathbf{4}$.

3 The DV Algorithm

Each node x maintains a **Distance Vector** $D_x = [D_x(y) : y \in N]$, which contains its current estimates of the least costs to all other nodes y.

Distance-Vector (DV) Algorithm for Node x

1. **Initialization:**
   - For all destinations y: $D_x(y) = c(x, y)$ (if not a neighbor, cost is ∞).
   - Send distance vector D_x to all neighbors.
2. **Loop (Forever):**
   1. **Wait** until a link cost change or an update from neighbor w arrives.
   2. **Update** distance vector using Bellman-Ford:
   $$D_x(y) = \min_v \{c(x, v) + D_v(y)\} \quad \text{for each } y \in N$$
   3. **If** $D_x(y)$ changed for any y, **notify** all neighbors by sending the new D_x.

4 Example Trace: Three-Node Network

Consider the simple triangle network **x** — **y** — **z** with the following costs: $c(x, y) = 2, c(y, z) = 1, c(x, z) = 7$.

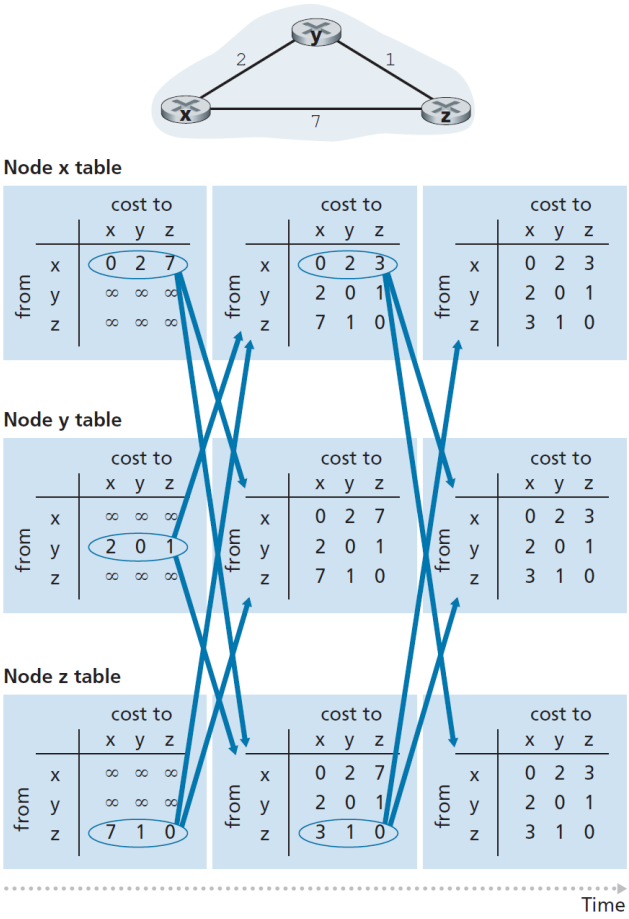

Figure 1: Distance-vector (DV) algorithm in operation

	Node x Table			Node y Table			Node z Table		
Time	**x**	**y**	**z**	**x**	**y**	**z**	**x**	**y**	**z**
Init	0	2	7	2	0	1	7	1	0
Step 1	0	2	**3**	2	0	1	**3**	1	0
Step 2	0	2	3	2	0	1	3	1	0

Table 1: Convergence of Distance Vectors. After Step 1, node x discovers a shorter path to z via y $(2 + 1 = 3)$.

Detailed Calculation for Node x at Step 1

When node x receives the distance vector $D_y = [2, 0, 1]$ from neighbor y:

- $D_x(x) = 0$
- $D_x(y) = \min\{c(x, y) + D_y(y), c(x, z) + D_z(y)\} = \min\{2 + 0, 7 + 1\} = 2$
- $D_x(z) = \min\{c(x, y) + D_y(z), c(x, z) + D_z(z)\} = \min\{2 + 1, 7 + 0\} = \mathbf{3}$

Since $D_x(z)$ changed from 7 to 3, node x updates its table and notifies its neighbors.

5 Dynamics: Link-Cost Changes

Good News vs. Bad News

- **Good News Travels Fast:** When a link cost decreases, the algorithm converges quickly across the network.
- **Bad News Travels Slowly:** When a link cost increases, the algorithm can suffer from **Routing Loops** and the **Count-to-Infinity** problem.

5.1 The Count-to-Infinity Problem

If a link cost increases significantly, nodes might mistakenly route through each other based on old information, gradually increasing their cost estimates by 1 in each iteration until they reach a very high threshold (or infinity).

5.2 Solution: Poisoned Reverse

Poisoned Reverse

If node z routes through node y to get to x, z will advertise to y that its distance to x is **infinity** ($D_z(x) = \infty$).

- This "little white lie" prevents y from trying to route to x via z, avoiding 2-node loops.
- **Limit:** It does not solve loops involving 3 or more nodes.

6 Comparison: LS vs. DV Algorithms

Attribute	**Link-State (LS)**	**Distance-Vector (DV)**
Message Complexity	$O(\|N\|\|E\|)$ messages; broadcast to all.	Exchanges only between neighbors.
Convergence Speed	Fast $O(N^2)$ calculation.	Varies; can be slow (count-to-infinity).
Robustness	Incorrect link costs isolated; nodes compute own tables.	Errors can propagate through the whole network via neighbor updates.

Table 2: Comparison of Routing Algorithm Attributes